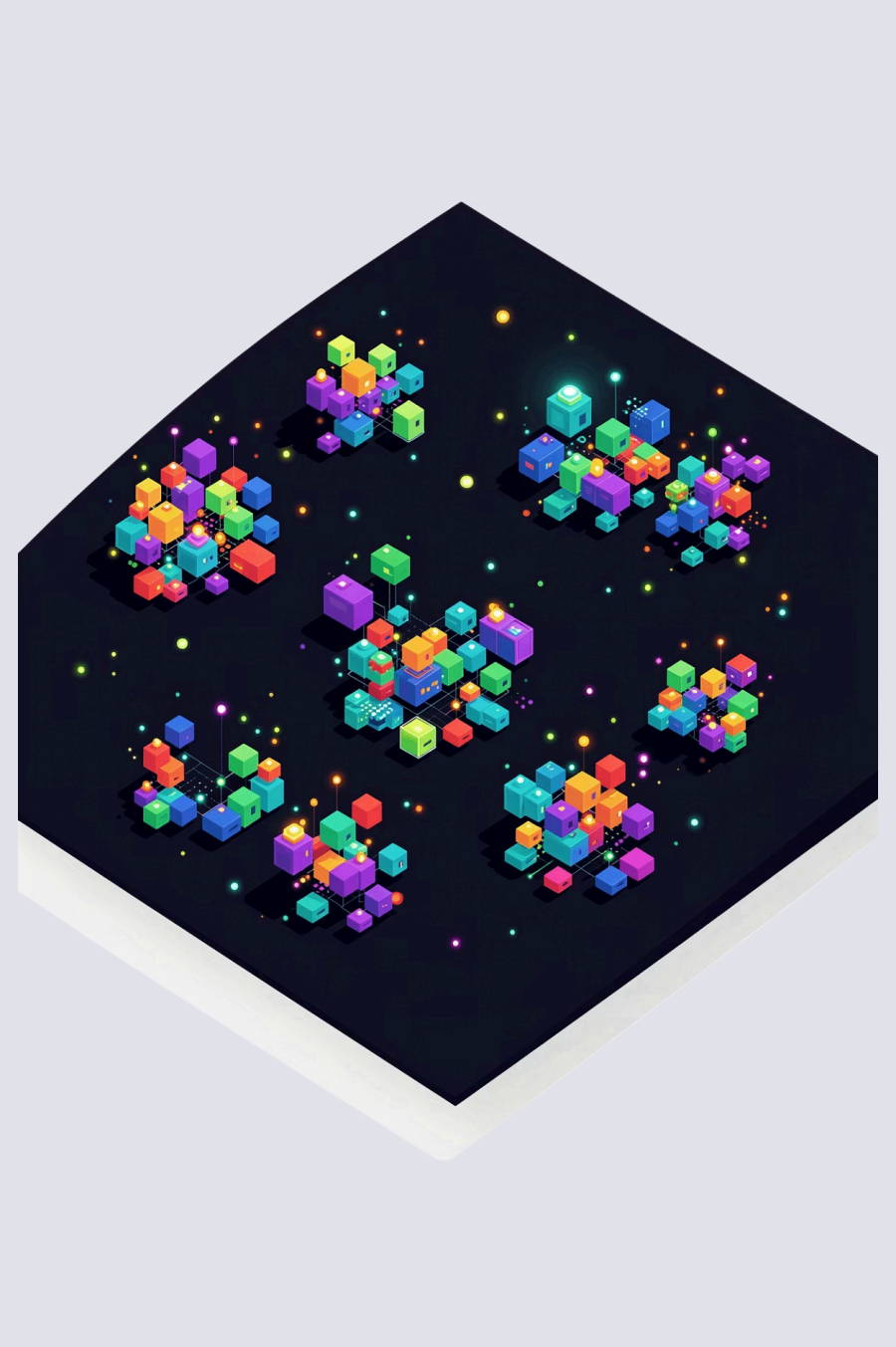




K-Means Clustering

An Introduction for Computer Engineering Students

Dr. A. V. Brahmane | Computer Engineering | Sanjivani College of Engineering

What is Clustering?

Clustering Defined

Clustering = Grouping similar data points together — without any labels!

Unsupervised Learning

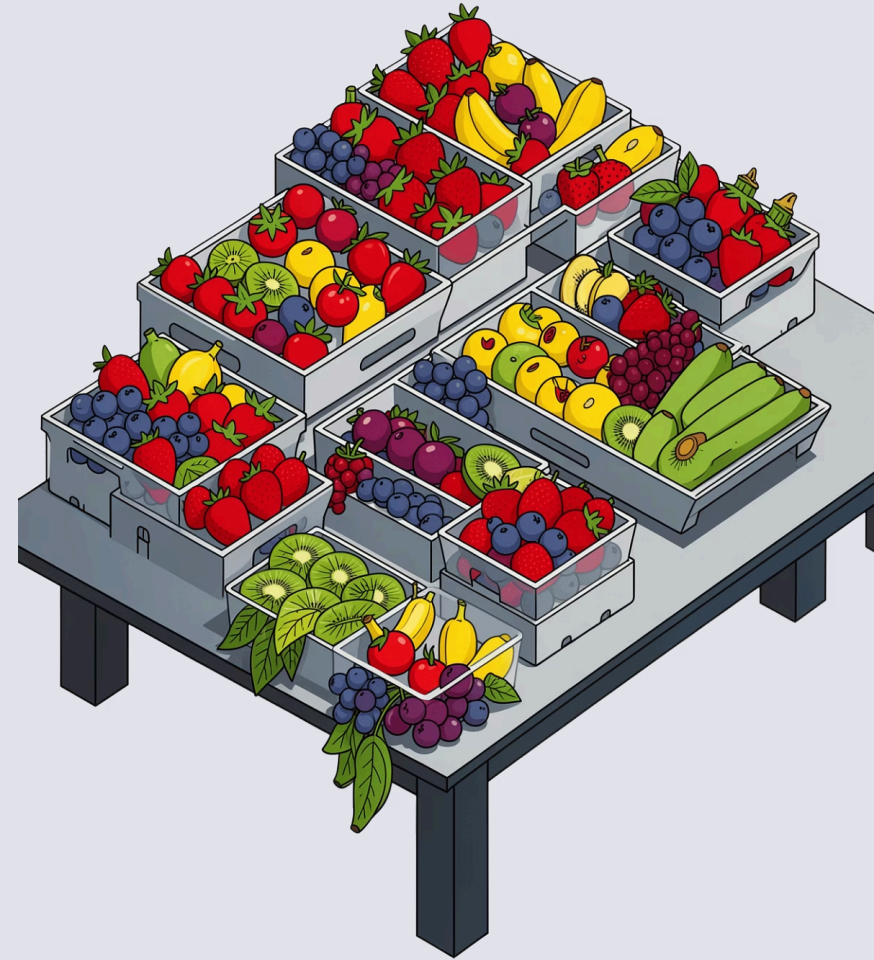
The algorithm finds patterns on its own — no human guidance needed.

Real-Life Analogy

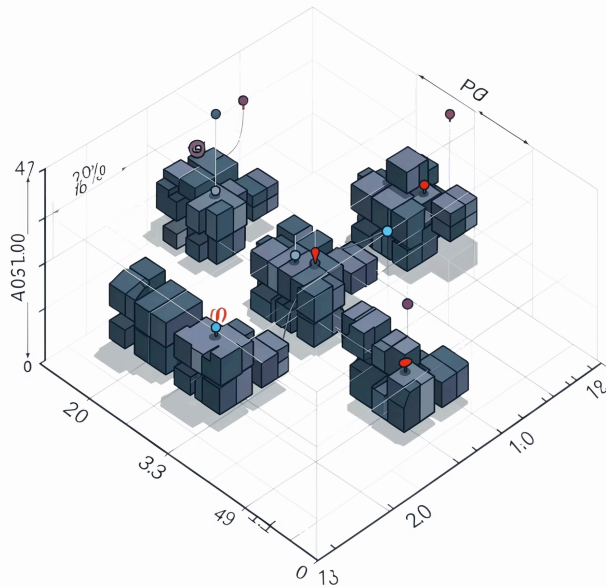
Sorting a bag of mixed fruits into groups — apples, bananas, oranges — by color and shape.

Most Popular Algorithm

K-Means is the most widely used clustering algorithm in Machine Learning.



What is K-Means Clustering?



Goal

Divide data into **K groups (clusters)** where points in the same group are as similar as possible.



Centroid

The center point of each cluster — K-Means keeps moving centroids until they settle.



Distance-Based

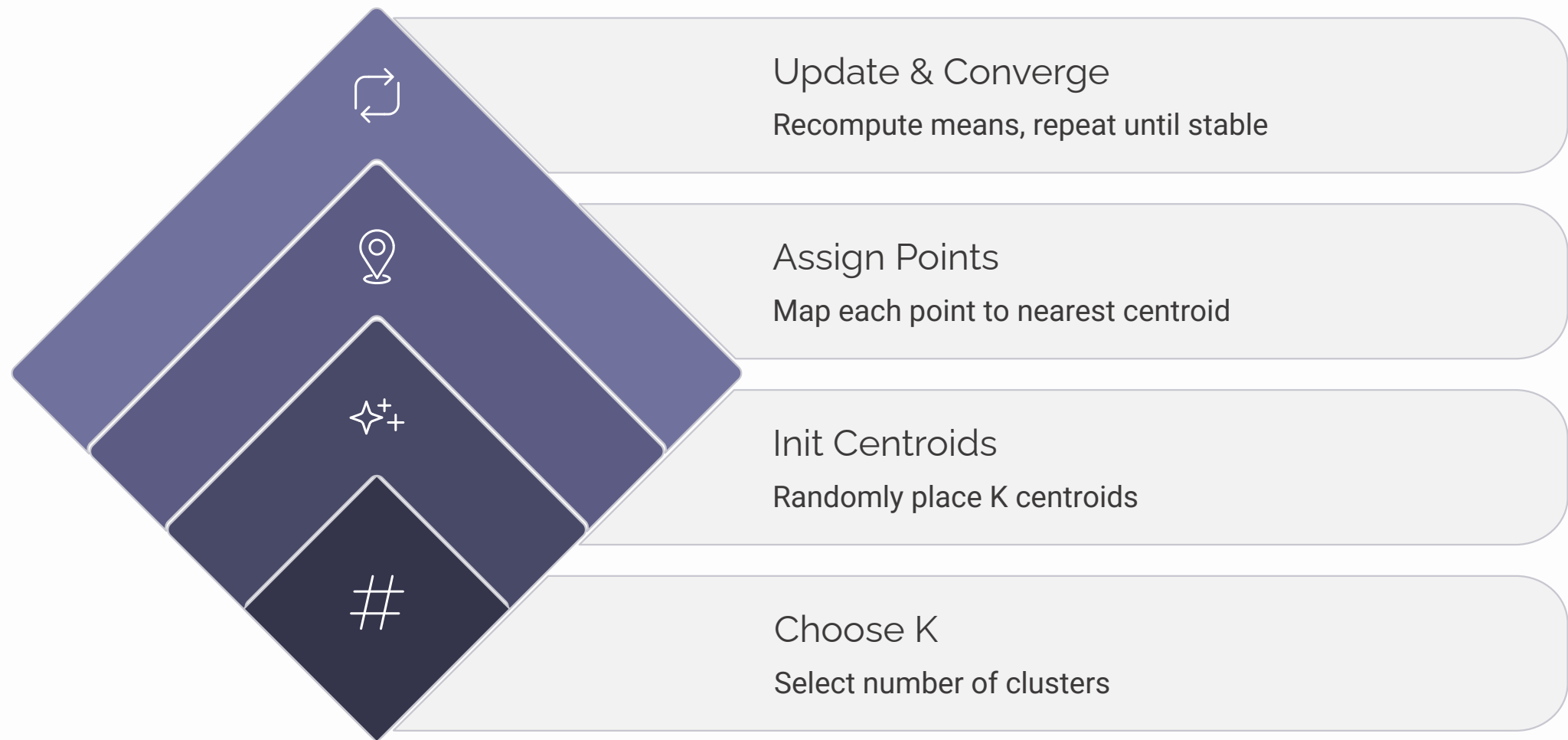
Uses **Euclidean distance** to decide which cluster a point belongs to.



Minimizes WCSS

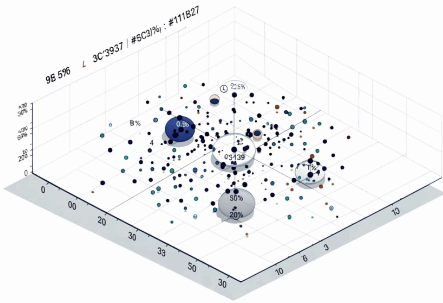
Minimizes **Within-Cluster Sum of Squares** — the total spread inside each cluster.

How K-Means Works — Step-by-Step Flowchart



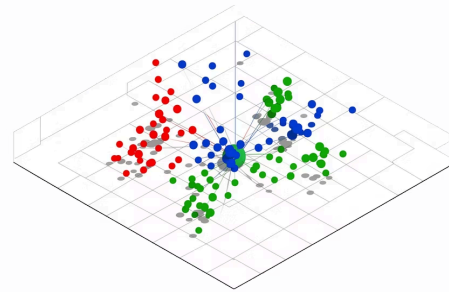
🔄 Steps 3–5 repeat until no point changes its cluster assignment — this is called **convergence**. The algorithm is guaranteed to converge in a finite number of iterations.

Visual Example — Iteration by Iteration



Iteration 1

Random centroids placed → points assigned to nearest centroid → 3 rough clusters form.



Iteration 2

Centroids move to the mean of each cluster → points reassigned → clusters improve.



Iteration 3 — Converged

Centroids barely move → algorithm converges → final clusters locked in. ☒

Python Implementation — Step by Step



01

Import Libraries

Import **numpy**, **matplotlib**, and **sklearn** — the three pillars of this implementation.

02

Generate Data

Use **make_blobs()** to create a simple 2D synthetic dataset with 3 natural clusters.

03

Apply K-Means

Instantiate **KMeans(n_clusters=3, random_state=42)** for reproducible results.

04

Fit & Predict

kmeans.fit_predict(X) assigns each data point to one of the K clusters.

05

Visualize

Scatter plot with color-coded clusters and red **X** markers for centroids.

Python Code — K-Means on 2D Data

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Step 1: Generate synthetic 2D dataset with 3 clusters
X, y = make_blobs(n_samples=300, centers=3,
                  cluster_std=0.8, random_state=42)

# Step 2: Apply K-Means Clustering
km = KMeans(n_clusters=3, init='k-means++',
            n_init=10, random_state=42)
labels = km.fit_predict(X)

# Step 3: Get centroid coordinates
centers = km.cluster_centers_

# Step 4: Visualize clusters
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels,
            cmap='viridis', s=50, alpha=0.7)
plt.scatter(centers[:, 0], centers[:, 1],
            marker='X', s=200, c='red', label='Centroids')
plt.title('K-Means Clustering (K=3)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()

print("Inertia (WCSS):", km.inertia_)
```

 `km.cluster_centers_` returns the (x, y) coordinates of each final centroid. `km.inertia_` gives the WCSS value directly.

Characteristics & Features of K-Means



Unsupervised

No labeled data needed — discovers hidden patterns automatically.



Scalable — $O(nk)$

Complexity scales as $O(nk)$ — works efficiently on large datasets.



Simple & Fast

Easy to implement and interpret; converges quickly in practice.



Sensitive to Init

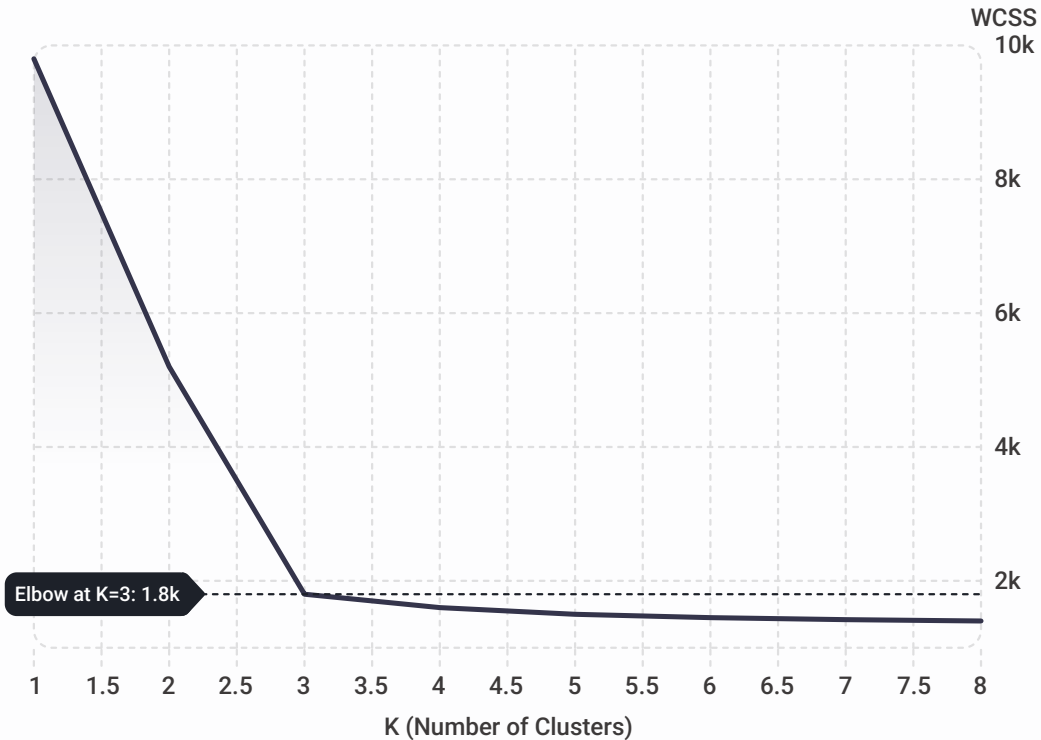
Different random starts can give different results — solved by K-Means++.



Spherical Clusters

Works best when clusters are roughly circular and similar in size.

The Elbow Method — Choosing the Best K



How the Elbow Method Works

Run K-Means for **K = 1 to 10** and record WCSS for each value of K.

- **Before the elbow:** WCSS drops quickly → clusters become much better
- **At the elbow:** The bend point = optimal K (here K=3)
- **After the elbow:** WCSS drops slowly → extra clusters add little value and may overfit

Python:

```
wcss.append(KMeans(n_clusters=i).fit(X).inertia_)
```

inside a loop, then plot WCSS vs K.

Performance Metrics for K-Means

WCSS / Inertia

Sum of squared distances from each point to its centroid.

Lower is better. Direct output:

```
km.inertia_
```

Silhouette Score

Measures how similar a point is to its own cluster vs. other clusters.

Ranges **-1 to +1** – closer to +1 means better separation. `silhouette_score(X, labels)`

Davies-Bouldin Index

Ratio of within-cluster scatter to between-cluster separation.

Lower is better.

```
davies_bouldin_score(X, labels)
```

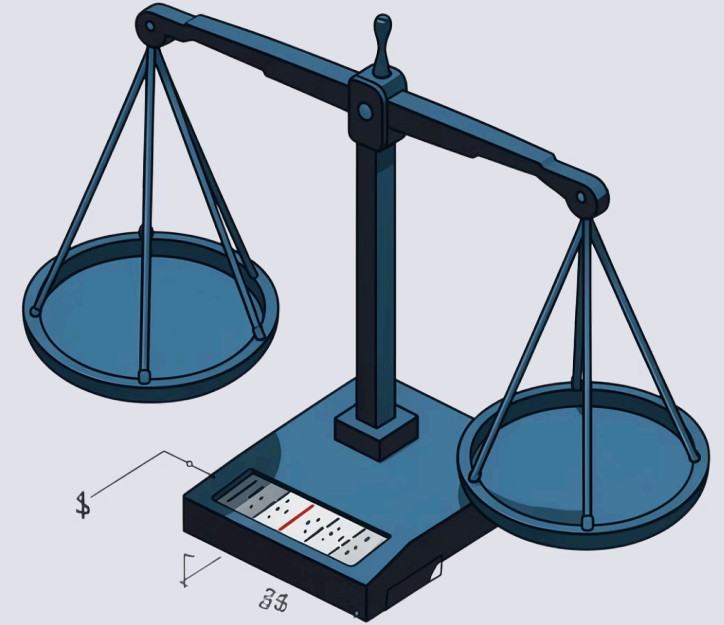
Advantages & Limitations

✓ Advantages

- **Simple to understand** and implement — great starting point for clustering
- **Efficient** on large datasets — scales linearly with data size $O(nk)$
- **Versatile** — used in customer segmentation, image compression, anomaly detection

✗ Limitations

- **Must specify K** in advance — not always obvious what K should be
- **Sensitive to outliers** — one bad point can pull a centroid far off
- **Fails on non-spherical clusters** — can't handle complex shapes like crescents or rings



Real-World Applications of K-Means



Customer Segmentation

Group shoppers by buying behavior for targeted marketing campaigns.

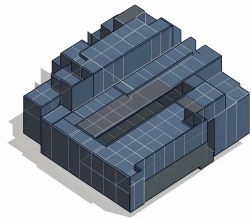


Image Compression

Reduce colors in an image by clustering similar pixel colors together.



Healthcare

Group patients by symptoms to identify disease patterns and risk profiles.



Document Clustering

Organize news articles or emails into topics automatically.



Geospatial Analysis

Cluster GPS locations to find hotspots or optimize delivery zones.

Key Takeaways

1

Cluster by Distance

K-Means groups data into **K clusters** by minimizing distance to centroids – simple, fast, powerful.

2

Iterative Process

Assign → Update → Repeat until **convergence**. Guaranteed to finish in finite steps.

3

Find Best K

Use the **Elbow Method** to find the optimal K; validate quality with **Silhouette Score**.

4

Easy in Python

`sklearn.cluster.KMeans` makes implementation straightforward in just a few lines of code.

❏ Best for spherical, well-separated clusters – always understand K-Means' limitations before applying it to real data!

Sources & References

KMeans Explained

ml-explained.com/blog/kmeans-explained

Google ML — What is K-Means Clustering?

developers.google.com/machine-learning/clustering/kmeans/overview

Elbow Method —
GeeksforGeeks

[geeksforgeeks.org — Elbow Method for Optimal K in KMeans](https://www.geeksforgeeks.org/elbow-method-for-optimal-k-in-kmeans/)